

<통신시스템 캡스톤 디자인1>

7주차 보고서

YOLO,DeepFace 판별 결과의
DB 저장 및 이벤트 알림 기능 구현

-오석준-

체크리스트

DAY	주요 계획	목표	비고
3.31	Raspberry Pi OS 기반 환경 점검 SSH 원격 접속 구성 네트워크 및 IP 확인 Python/pip/venv 환경 구성, YOLO 설치 및 실행 가능성 검토	라즈베리파이 개발 환경 구축 완료 YOLO 실행 오류 (Illegal instruction) 확인	완료
4.7	PC 기반 FastAPI 서버 구축 기본 API(GET/POST) 작성 라즈베리파이↔서버 통신 확인	라즈베리파이와 PC 서버 간 통신 가능 상태 확보	완료
4.14	USB 웹캠(Logitech C920) 연결 카메라 인식 확인 OpenCV 기반 영상 캡처 및 이미지 저장	영상 입력 환경 구축 완료	완료
	라즈베리파이에서 캡처한 이미지를 PC 서버로 전송 HTTP POST 방식 업로드 기능 구현 라이브 스트리밍 기능 구현	이미지 전송 기능 구현	
5.8	PC 서버에서 YOLOv8n/v8s 실행 환경 구성 이미지 기반 객체 탐지 테스트	객체 탐지 기능 1차 구현	완료
5.19	등록 얼굴 데이터 수집, 얼굴 검출 및 특 정 추출 방식 설계, 등록 인물과 미등록 인물을 구분하기 위한 얼굴 인식 구조 설계, 결과 저장을 위한 DB 구조 설계	얼굴 인식 엔진 및 데 이터 저장 체계 확립	완료
	Opencv 기반 얼굴 검출과 특징 비교 기 능 구현, 등록 인물/미등록 인물 판별 테 스트, yolo 사람 탐지 결과와 얼굴 인식 결과를 결합한 판별하는 로직을 구현	등록 인물과 외부인 구 분 기능 구현, 객체 탐 지와 얼굴 인식을 결합 한 판별 구조 확인	
5.26	이벤트 정의 및 알람 기능 구현 침입 발생 시 로그/알람 생성	이벤트 기반 시스템 완성	완료
6.2	전체 시스템 통합 테스트 오류 수정 및 성능 개선	1학기 1차 완성본 도출	

목 차

1. 7주차 수행 개요
2. 6주차 대비 개선 사항
 - 2.1 기존 처리 방식의 한계
 - 2.2 등록 얼굴 임베딩 사전 캐싱 적용
 - 2.3 NumPy 기반 cosine distance 직접 비교
 - 2.4 Early Exit 구조 적용
 - 2.5 처리 속도 개선 결과
3. 이벤트 정의 및 분류 구조
 - 3.1 이벤트 종류 정의
 - 3.2 final_result → event_type 매핑 로직
4. MySQL DB 연동 구현
 - 4.1 detection_log 테이블 확장
 - 4.2 PyMySQL 기반 자동 저장 로직
 - 4.3 저장 결과 확인
5. 실시간 알림 시스템 구축
 - 5.1 알림 채널 비교 분석
 - 5.2 텔레그램 봇과 이메일 선정 사유
 - 5.3 텔레그램 봇 구축 과정
 - 5.4 이메일 알림 기능 추가
 - 5.5 이메일·텔레그램 이중 알림 전송 로직 구현
6. 통합 시스템 동작 흐름
7. 실험 결과
 - 7.1 등록 인물 인식 결과
 - 7.2 미등록 인물 탐지 결과
 - 7.3 얼굴 식별 실패 및 보수적 판별 결과
 - 7.4 실험 결과 종합

8. 7주차 결과 및 한계

9. 8주차 계획

1. 7주차 수행 개요

7주차에서는 6주차에 구현한 YOLO와 DeepFace 결합 구조를 기반으로, 단순히 탐지 결과를 출력하는 단계를 넘어 이벤트 기반 시스템으로 확장하는 작업을 수행하였다.

먼저 DeepFace 판별 결과인 `final_result` 값에 따라 `NORMAL_ACCESS`, `UNKNOWN_PERSON_DETECTED`, `FACE_NOT_IDENTIFIED` 등 6가지 이벤트로

분류하는 구조를 설계하였다. 이후 모든 탐지 결과를 MySQL `detection_log` 테이블에 자동으로 저장하도록 PyMySQL 기반 저장 함수를 구현하였으며, 미등록 인물이 감지되었을 때 관리자에게 즉시 알림을 전송하기 위해 텔레그램 봇과 이메일 기반 이중 알림 시스템을 구축하였다.

또한, 6주차 코드의 반복적인 `DeepFace.verify()` 호출로 인한 처리 속도 한계를 분석하여, 등록 얼굴 임베딩 사전 캐싱, NumPy 기반 cosine distance 직접 비교, Early Exit 구조를 적용함으로써 전체 처리 속도를 개선하였다.

2. 6주차 구조 개선 사항

2.1 기존 처리 방식의 한계

6주차 코드에서는 YOLO로 사람을 탐지한 뒤, 탐지된 사람 영역을 crop 이미지로 저장하고 DeepFace를 이용하여 등록 얼굴 이미지와 비교하였다. 이때 입력 이미지마다 등록 얼굴 이미지 전체를 대상으로 DeepFace.verify()를 반복 수행하는 구조였기 때문에, 등록 이미지 수가 많아질수록 처리 시간이 증가하는 한계가 있었다.

또한 full crop과 upper crop을 각각 비교하고, 여러 detector_backend를 순차적으로 적용하는 과정에서 동일한 등록 얼굴 이미지의 특징값을 반복적으로 추출하게 되어 불필요한 연산이 발생하였다. 따라서 7주차에서는 얼굴 인식 정확도는 유지하면서 반복 연산을 줄이는 방향으로 처리 속도 개선을 진행하였다.

2.2 등록 얼굴 임베딩 사전 캐싱 적용

처리 속도를 개선하기 위해 등록 얼굴 이미지를 프로그램 실행 초기에 한 번만 분석하여 임베딩 값을 미리 저장하는 방식을 적용하였다. 기존 방식은 테스트 이미지가 들어올 때마다 등록 얼굴 이미지를 반복적으로 불러와 DeepFace 비교를 수행하였지만, 개선 방식에서는 등록 얼굴의 임베딩을 사전에 추출하여 메모리에 캐싱하였다.

```
def precompute_registered_embeddings(registered_dir):
    cache = {detector: [] for detector in DETECTOR_BACKENDS}
    reg_files = get_image_list(registered_dir)
    for reg_path in reg_files:
        for detector in DETECTOR_BACKENDS:
            rep = DeepFace.represent(
                img_path=str(reg_path),
                model_name=MODEL_NAME,
                detector_backend=detector,
                enforce_detection=False,
                align=True
            )
```

```

        cache[detector].append({
            "name": reg_path.name,
            "embedding": rep[0]["embedding"]
        })
    return cache

```

이 구조를 적용하면 등록 얼굴 이미지는 프로그램 시작 시 1회만 임베딩되고, 이후 테스트 이미지에서는 입력 얼굴의 임베딩만 추출하여 기존에 저장된 등록 얼굴 임베딩과 비교하면 된다. 이를 통해 DeepFace 연산 횟수를 줄이고 전체 처리 시간을 단축할 수 있었다.

2.3 NumPy 기반 cosine distance 직접 비교

기존에는 DeepFace.verify()를 사용하여 입력 얼굴과 등록 얼굴을 비교하였다. 그러나 이 방식은 비교할 때마다 내부적으로 얼굴 검출, 정렬, 임베딩 추출, distance 계산을 반복 수행하기 때문에 처리 시간이 증가한다.

7주차 개선 코드에서는 입력 얼굴의 임베딩을 추출한 뒤, 캐싱된 등록 얼굴 임베딩과 직접 cosine distance를 계산하도록 변경하였다.

```

def cosine_distance(a, b):
    a = np.asarray(a, dtype=np.float32)
    b = np.asarray(b, dtype=np.float32)
    return 1.0 - np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)
    + 1e-8)

```

이를 통해 등록 얼굴 이미지에 대한 반복적인 DeepFace 호출을 줄이고, 비교 연산은 NumPy 기반 벡터 연산으로 처리하였다. NumPy 연산은 단순 거리 계산에 적합하고 처리 속도가 빠르기 때문에, 등록 얼굴 수가 증가할수록 기존 방식보다 효율적인 구조가 된다.

출처: [코사인 유사도와 python 구현](#)

2.4 Early Exit 구조 적용

추가적으로, 비교 과정에서 매우 확실한 등록 인물 매칭이 발생하면 더 이상 나머지 등록 얼굴과 비교하지 않고 즉시 결과를 반환하는 Early Exit 구조를 적용하였다.

```
EARLY_EXIT_DISTANCE = 0.30
```

```
if best["best_distance"] is not None and best["best_distance"] < EARLY_EXIT_DISTANCE:
```

```
    return {**best, "final_result": "Registered"}
```

본 시스템에서는 distance 값이 0.30 미만이면 등록 인물과 매우 유사한 것으로 판단하고, 추가 비교를 생략하도록 하였다. 이 방식은 등록 인물임이 명확한 경우 불필요한 반복 비교를 줄여 전체 처리 속도를 개선하는 데 효과적이다.

2.5 처리 속도 개선 결과

7주차 개선을 통해 기존의 반복적인 DeepFace.verify() 중심 비교 방식에서 벗어나, 등록 얼굴 임베딩 캐싱과 NumPy 기반 cosine distance 비교 방식으로 구조를 변경하였다. 또한 Early Exit를 적용하여 확실한 매칭이 발생한 경우 비교를 조기에 종료하도록 하였다.

개선 전후 구조를 비교하면 다음과 같다.

처리 단계	6주차 기존 방식	7주차 개선 방식
등록 얼굴 임베딩 추출	매 호출 시 반복 수행	시작 시 1회 캐싱
얼굴 비교 방식	DeepFace.verify() 반복 호출	NumPy cosine distance 직접 계산
1 이미지당 DeepFace 호출 횟수	약 20~40회	약 1~2회
확실한 매칭 시 처리	모든 비교 계속 수행	Early Exit로 즉시 종료
전체 처리 시간 (등록 얼굴 10장 기준)	약 30~60분	약 5~10분
개선율	-	약 5~10배 단축

처리 단계별 평균 소요 시간

처리 단계	평균 소요 시간
YOLO 객체 탐지	50 ~ 200 ms
DeepFace 얼굴 임베딩 (RetinaFace)	500 ~ 2,000 ms
DeepFace 얼굴 임베딩 (OpenCV)	100 ~ 300 ms
NumPy cosine distance 계산	1 ms 미만
MySQL INSERT	5 ~ 20 ms
Telegram 알림 전송	300 ~ 1,000 ms
이메일 알림 전송	1,000 ~ 3,000 ms

따라서 7주차에서는 얼굴 인식 구조의 정확도는 유지하면서도, 등록 얼굴 임베딩을 재사용하고 불필요한 반복 비교를 줄이는 방식으로 처리 속도를 개선하였다. 이 개선은 향후 등록 인물 수가 증가하거나 라즈베리파이 실시간 영상 입력 환경으로 확장할 때 더 큰 효과를 발휘할 것으로 판단된다.

3. 이벤트 정의 및 분류 구조

3.1 이벤트 종류 정의

DeepFace 판별 결과인 final_result를 기반으로, 시스템 동작 관점에서 의미가 있는 6가지 이벤트로 분류하였다. 이벤트는 단순한 판별 결과를 넘어, 알림이 필요한 상황인지 아닌지를 함께 표현하기 위해 도입하였다.

final_result	event_type	alert_required	설명
Registered	NORMAL_ACCESS	False	등록 인물 정상 출입
Unknown	UNKNOWN_PERSON_DETECTED	True	미등록 인물 감지
Unidentified	FACE_NOT_IDENTIFIED	True	사람은 있으나 얼굴 식별 불가
No person detected	NO_PERSON	False	사람 미감지
Crop failed	CROP_FAILED	True	사람 탐지 후 crop 실패
No registered face data	NO_REGISTERED_FACE_DATA	True	등록 데이터 없음

alert_required가 True인 이벤트는 텔레그램 알림과 로그 파일 저장 대상이며, False인 이벤트는 DB 기록만 수행한다. 이러한 구조를 통해 정상 출입 시에는 불필요한 알림을 발생시키지 않고, 보안상 의심되는 상황에서만 관리자에게 알림이 도달하도록 설계하였다.

3.2 final_result에서 event_type으로의 매핑

get_event_info() 함수를 통해 final_result에 해당하는 event_type, alert_required, alert_message를 한 번에 반환하도록 구현하였다. 매핑 함수를 분리함으로써 추후 새로운 이벤트가 추가되더라도 함수 내부만 수정하면 되도록 확장성을 확보하였다.

#이벤트 매핑 함수

```
def get_event_info(final_result):
    if final_result == "Registered":
        return {
            "event_type": "NORMAL_ACCESS",
            "alert_required": False,
            "alert_message": "Registered person detected."
        }
    elif final_result == "Unknown":
        return {
            "event_type": "UNKNOWN_PERSON_DETECTED",
            "alert_required": True,
            "alert_message": "Alert: Unknown person detected."
        }
    elif final_result == "Unidentified":
        return {
            "event_type": "FACE_NOT_IDENTIFIED",
            "alert_required": True,
            "alert_message": "Alert: Person detected, but face could
not be identified."
        }
    # 뒤는 동일함.
```

4. MySQL DB 연동 구현

4.1 detection_log 테이블 확장

6주차에서 설계한 detection_log 테이블에 이벤트 정보, 알림 정보, 결과 이미지 경로 컬럼을 추가하였다. 이는 단순 판별 결과뿐 아니라 시스템이 어떤 이벤트로 처리하였고 어떤 알림이 발생하였는지까지 추적할 수 있도록 하기 위함이다.

추가된 컬럼은 best_verified, best_detector, best_crop_type, crop_path, annotated_path, event_type, alert_required, alert_message 8개이다. best_detector와 best_crop_type은 어떤 detector와 어떤 crop 종류에서 매칭이 성공했는지를 기록하여, 추후 어떤 입력 조건이 가장 효과적이었는지 분석할 수 있도록 한다.

```
CREATE TABLE detection_log (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    detected_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    test_group VARCHAR(50),  
    model_name VARCHAR(20),  
    image_path VARCHAR(255),  
    person_index INT,  
    person_detected BOOLEAN,  
    yolo_confidence FLOAT,  
    yolo_inference_ms FLOAT,  
    recognized_name VARCHAR(100),  
    face_distance FLOAT,  
    best_verified BOOLEAN,  
    best_detector VARCHAR(30),  
    best_crop_type VARCHAR(20),  
    is_registered BOOLEAN,  
    final_result VARCHAR(50),  
    crop_path VARCHAR(255),  
    annotated_path VARCHAR(255),  
    event_type VARCHAR(50),  
    alert_required BOOLEAN,
```

```
        alert_message VARCHAR(255)
    );
```

4.2 PyMySQL 기반 자동 저장 로직

판별 결과 row 1건마다 detection_log 테이블에 INSERT하는 함수 save_detection_log_to_db()를 구현하였다. DB 접속은 매 INSERT마다 새로 연결하고 종료하는 방식으로 구성하여, 장기 실행 시 연결이 끊어지는 문제를 회피하였다.

#DB 접속 정보

DB_HOST = "127.0.0.1"

DB_PORT = 3306

DB_USER = "root"

DB_PASSWORD = "<DB 비밀번호>"

DB_NAME = "capstone_db"

#DB 저장 함수

```
def save_detection_log_to_db(row):
    if not ENABLE_DB_SAVE:
        return
    conn = None
    try:
        conn = get_db_connection()
        with conn.cursor() as cursor:
            sql = """
INSERT INTO detection_log (
test_group, model_name, image_path, person_index,
person_detected, yolo_confidence, yolo_inference_ms,
recognized_name, face_distance, best_verified,
best_detector, best_crop_type, is_registered,
final_result, crop_path, annotated_path,
event_type, alert_required, alert_message
)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
        %s, %s, %s, %s, %s, %s, %s, %s, %s)
"""
            final_result = row["final_result"]
```

```

        person_detected =final_result != "No person detected"
        is_registered =final_result == "Registered"
        cursor.execute(sql, (
            row["test_group"], row["model_name"], row["image_name"],
            row["person_index"], person_detected,
row["yolo_confidence"],
            row["yolo_inference_ms"], row["best_match"],
row["best_distance"],
            row["best_verified"], row["best_detector"],
row["best_crop_type"],
            is_registered, row["final_result"], row["crop_path"],
            row["annotated_path"], row["event_type"],
            row["alert_required"], row["alert_message"]
        ))

        conn.commit()
    except Exception as e:
        print("DB 저장 오류:", e)
    finally:
        if conn is not None:
            conn.close()

```

출처: [pymysql 사용법](#) , [pymsql 사용법 2](#)

4.3 저장 결과 확인

전체 테스트 종료 후 MySQL Workbench에서 SELECT 쿼리로 저장 결과를 확인하였다. 각 이미지별로 YOLO 신뢰도, DeepFace 거리, 최종 판별 결과, 이벤트 종류, 알림 필요 여부가 정상적으로 기록되었음을 확인할 수 있다.

```
SELECT final_result, COUNT(*) AS 건수, AVG(best_distance) AS 평균거리
```

```
FROM detection_log
```

```
GROUP BY final_result;
```



The screenshot shows the MySQL Workbench interface. The SQL editor at the top contains the following query:

```
SELECT  
  final_result,  
  COUNT(*) AS 건수,  
  AVG(best_distance) AS 평균거리  
FROM detection_log  
GROUP BY final_result;
```

Below the editor, the 'Result Grid' tab is active, displaying the query results in a table with three columns: final_result, 건수, and 평균거리. The results are grouped by final_result.

final_result	건수	평균거리
NoPerson	16	0.018765133187367486
Registered	58	0.018765133187367486

이 쿼리를 통해 Registered, Unknown별 처리 건수와 평균 distance를 한 번에 확인할 수 있어, 시스템 동작의 통계적 분석 기반을 마련하였다.

5. 실시간 알림 시스템 구축

5.1 알림 채널 비교 분석

미등록 인물 감지 시 관리자에게 즉시 알림을 전송하기 위한 채널을 비교 분석하였다. 검토 대상은 이메일, 텔레그램, 카카오톡, 디스코드 4가지이다.

채널	구현 난이도	응답 속도	이미지 첨부	선정 여부	선정 사유
이 메 일 SMTP	낮음	보통	가능	O	상세 정보 기록 및 사후 확인에 유리
텔 레 그 램 Bot	낮음	빠름	가능	O	모바일 즉시 알림 및 이미지 전송에 적합
카 카 오 톡 알림톡	높음	빠름	제한적	X	사업자 등록 및 승인 절차 필요
디 스 코 드 Webhook	낮음	빠름	가능	X	사용자가 앱을 상시 확인하지 않을 수 있음

5.2 텔레그램 봇과 이메일 선정 사유

미등록 인물 또는 얼굴 식별 실패 상황이 발생했을 때 관리자에게 신속하게 알림을 전달하기 위해 이메일, 텔레그램, 카카오톡, 디스코드 알림 방식을 비교하였다. 카카오톡 알림톡은 국내 사용성이 높지만 사업자 등록, 채널 승인, 템플릿 심사 등의 절차가 필요하여 캡스톤 프로젝트에 적용하기에는 구현 부담이 크다고 판단하였다. 디스코드 Webhook은 구현이 간단하지만, 사용자가 디스코드 앱을 상시 확인하지 않을 경우 즉시성이 떨어질 수 있다.

이메일은 SMTP를 이용하면 구현이 비교적 간단하고, 이벤트 내용이 메일함에 기록으로 남아 사후 확인에 유리하다. 특히 이벤트 유형, 탐지 이미지명, YOLO confidence, DeepFace distance, 결과 이미지 경로 등 상세 정보를 전달하기에 적합하다. 다만 수신 환경에 따라 알림 도착이 지연될 수 있다는 한계가 있다.

텔레그램은 BotFather를 통해 봇을 쉽게 생성할 수 있고, 별도의 승인 절차 없이 무료로 사용할 수 있다. 또한 sendPhoto API를 이용하면 탐지 결과가 표시된 이미지를 함께 전송할 수 있어, 관리자가 모바일 환경에서 이상 상황을 빠르게 확인할 수 있다.

따라서 본 프로젝트에서는 이메일과 텔레그램을 함께 사용하는 이중 알림 구조를 선정하였다. 이메일은 기록성과 상세 정보 전달을 담당하고, 텔레그램은 즉시성과 이미지 기반 확인을 담당하도록 구성하였다. 이를 통해 미등록 인물 감지 상황에 대한 알림 누락 가능성을 줄이고, 시스템의 대응성을 높이하고자 하였다.

5.3 텔레그램 봇 구축 과정

텔레그램 봇 구축은 다음 4단계로 진행하였다.

(1) Telegram 앱에서 BotFather를 검색하여 1:1 대화를 시작하고, /newbot 명령으로 새 봇을 생성하였다. 봇 표시 이름과 사용자 이름(끝이 _bot으로 끝나야 함)을 입력하면 BotFather가 봇 토큰을 발급한다.

(2) 발급받은 봇과 1:1 대화를 시작하고, /start 명령 또는 임의의 메시지를 전송하였다. 이 단계가 선행되어야 봇이 사용자의 chat_id를 인식한다.

(3) 웹 브라우저에서 https://api.telegram.org/bot<발급받은_토큰>/getUpdates URL을 호출하여 응답 JSON을 확인하였다. 응답 내 "chat":{"id": <숫자>} 필드의 값이 본인의 chat_id이다.

(4) 발급받은 봇 토큰과 chat_id를 Python 코드의 설정 변수에 등록하였다.

```
ENABLE_TELEGRAM_ALERT = True
TELEGRAM_BOT_TOKEN = "<발급받은 봇 토큰>"
TELEGRAM_CHAT_ID = "<본인 chat_id>"
TELEGRAM_SEND_PHOTO = True

def send_telegram_alert(event_info, row):
    if not ENABLE_TELEGRAM_ALERT or not event_info["alert_required"]:
        return

    caption = (
        f"[{event_info['event_type']}]\\n"
        f"{event_info['alert_message']}\\n"
        f"이미지: {row['image_path']}\\n"
        f"distance: {row['face_distance']}"
    )
```

```

if TELEGRAM_SEND_PHOTO and row["annotated_path"]:
    url =
    f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendPhoto"
    with open(row["annotated_path"], "rb") as f:
        requests.post(url,
            data={"chat_id": TELEGRAM_CHAT_ID, "caption":
caption},
            files={"photo": f})
else:
    url =
    f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    requests.post(url, data={"chat_id": TELEGRAM_CHAT_ID, "text":
caption})

```

BOTFATHER



출처: [텔레그램, 텔레그램으로 파이썬 알림 봇 생성 후 api 사용하기](#)

5.4 이메일 알림 기능 추가

텔레그램 알림과 함께 Gmail SMTP 기반 이메일 알림 기능을 추가하였다. 이메일 알림은 미등록 인물 감지, 얼굴 식별 실패, crop 실패 등 alert_required가 True로 분류된 이벤트가 발생했을 때 자동으로 전송되도록 구현하였다.

이메일 알림에는 이벤트 유형, 최종 판별 결과, 테스트 이미지명, 사용 모델, YOLO confidence, YOLO 처리 시간, DeepFace distance, detector, crop type, 결과 이미지 경로 등이 포함되도록 구성하였다. 이를 통해 관리자는 단순히 알림 발생 여부만 확인하는 것이 아니라, 어떤 이미지에서 어떤 조건으로 이상 상황이 발생했는지 구체적으로 확인할 수 있다.

이메일 알림 기능은 텔레그램 알림과 비교했을 때 즉시성은 다소 낮을 수 있으나, 알림 내용이 메일함에 기록으로 남기 때문에 사후 확인과 결과 증빙에 유리하다. 따라서 본 시스템에서는 이메일을 상세 정보 기록용 알림 채널로 활용하고, 텔레그램은 모바일 즉시 확인용 알림 채널로 활용하도록 구성하였다.

이메일 전송은 Python의 smtplib와 email.mime 모듈을 이용하여 구현하였다. Gmail 계정 사용 시 일반 비밀번호가 아닌 Google 계정에서 발급받은 앱 비밀번호를 사용해야 하며, SMTP 서버는 smtp.gmail.com, 포트는 587을 사용하였다.

```
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587
EMAIL_SENDER = "<보내는 이메일>"
EMAIL_APP_PASSWORD = "<앱 비밀번호>"
EMAIL_RECEIVER = "<받는 이메일>"
```

```
def send_email_alert(event_info, row):
    if not ENABLE_EMAIL_ALERT or not event_info["alert_required"]:
        return

    subject = f"[Capstone Alert] {event_info['event_type']}"
    body = f"""
```

침입 감지 시스템 알림

[이벤트 정보]

이벤트 종류 : {event_info['event_type']}
알림 메시지 : {event_info['alert_message']}
최종 판별 결과 : {row['final_result']}

[탐지 정보]

이미지명 : {row['image_path']}
사용 모델 : {row['model_name']}
YOLO 신뢰도 : {row['yolo_confidence']}
DeepFace 거리 : {row['face_distance']}
결과 이미지 : {row['annotated_path']}

"""

```
msg = MIMEMultipart()
msg["From"] = EMAIL_SENDER
msg["To"] = EMAIL_RECEIVER
msg["Subject"] = subject
msg.attach(MIMEText(body, "plain", "utf-8"))

server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT, timeout=20)
server.starttls()
server.login(EMAIL_SENDER, EMAIL_APP_PASSWORD)
server.sendmail(EMAIL_SENDER, EMAIL_RECEIVER,
msg.as_string())
server.quit()
```

최종적으로 process_result_row() 함수에서 이벤트 정보를 생성한 뒤, 알림이 필요한 경우 이메일 알림과 텔레그램 알림이 순차적으로 실행되도록 구성하였다. 이를 통해 이상 상황 발생 시 기록성과 즉시성을 모두 확보할 수 있는 이중 알림 구조를 구현하였다.

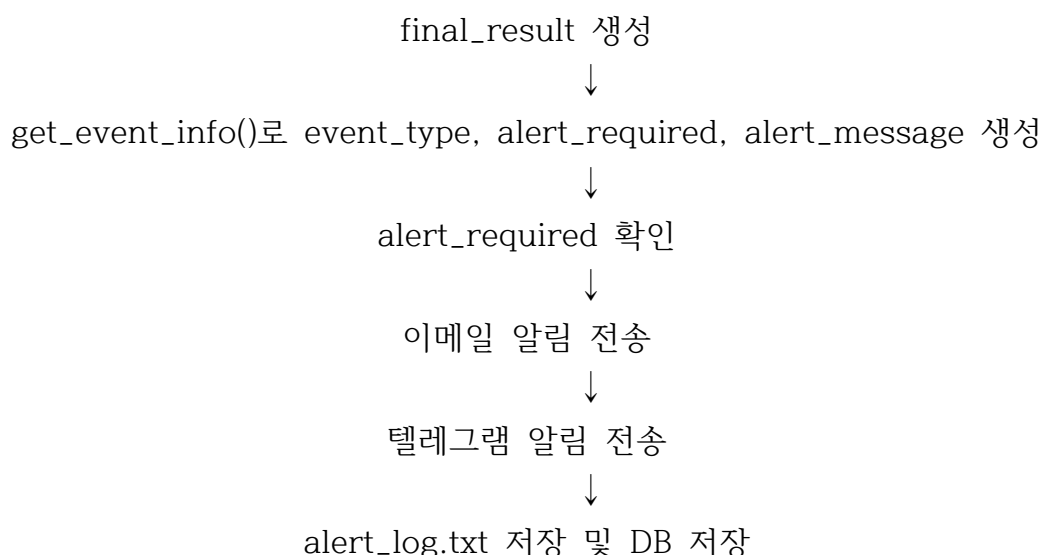
출처: [SMTP](#), [smtplib](#), [smtplib 사용예시](#)

5.5 이메일·텔레그램 이중 알림 전송 로직 구현

본 시스템에서는 `final_result` 값에 따라 이벤트 정보를 생성한 뒤, `alert_required`가 `True`인 경우 이메일과 텔레그램 알림을 순차적으로 전송하도록 구현하였다. 이를 위해 먼저 `get_event_info()` 함수에서 최종 판별 결과를 이벤트 유형으로 변환하고, 해당 이벤트가 알림 대상인지 판단하도록 구성하였다.

알림 대상 이벤트는 `Unknown`, `Unidentified`, `Crop failed`, `No registered face data`와 같이 보안상 확인이 필요한 경우이다. 반면 `Registered`와 `No person detected`는 정상 접근 또는 사람 미탐지 상황이므로 알림을 발생시키지 않고 DB와 CSV에만 기록되도록 하였다.

이메일 알림은 이벤트 유형, 최종 판별 결과, 테스트 이미지명, 사용 모델, YOLO confidence, DeepFace distance, 결과 이미지 경로 등 상세 정보를 텍스트 형태로 전송한다. 이를 통해 관리자는 이상 상황의 세부 내용을 메일함에서 기록으로 확인할 수 있다. 텔레그램 알림은 `annotated` 이미지가 존재하는 경우 결과 이미지를 함께 전송하여, 관리자가 모바일 환경에서 탐지 결과를 즉시 시각적으로 확인할 수 있도록 하였다.



결과 처리 함수

```
def process_result_row(row):  
    event_info = get_event_info(row["final_result"])  
  
    row["event_type"] = event_info["event_type"]  
    row["alert_required"] = event_info["alert_required"]  
    row["alert_message"] = event_info["alert_message"]  
  
    send_alert(event_info, row)  
    save_alert_log(event_info, row)  
    send_email_alert(event_info, row)  
    send_telegram_alert(event_info, row)  
    save_detection_log_to_db(row)  
  
    return row
```

이메일

[Capstone Alert] UNKNOWN_PERSON_DETECTED



받은편지함 ✕



석준

lose84681221@gmail.com

오전 11:04 (5시간 전)



나에게 ▼

침입 감지 시스템 알림

[이벤트 정보]

이벤트 종류 : UNKNOWN_PERSON_DETECTED

알림 메시지 : Alert: Unknown person detected.

최종 판별 결과 : Unknown

[탐지 정보]

테스트 그룹 : unknown_person

이미지명 : person_side_far_01.jpg

사용 모델 : yolov8s

YOLO confidence: 0.9441

YOLO 처리 시간 : 364.19 ms

[얼굴 인식 정보]

DeepFace distance: 0.63919997215271

Detector : retinaface

Crop type : upper

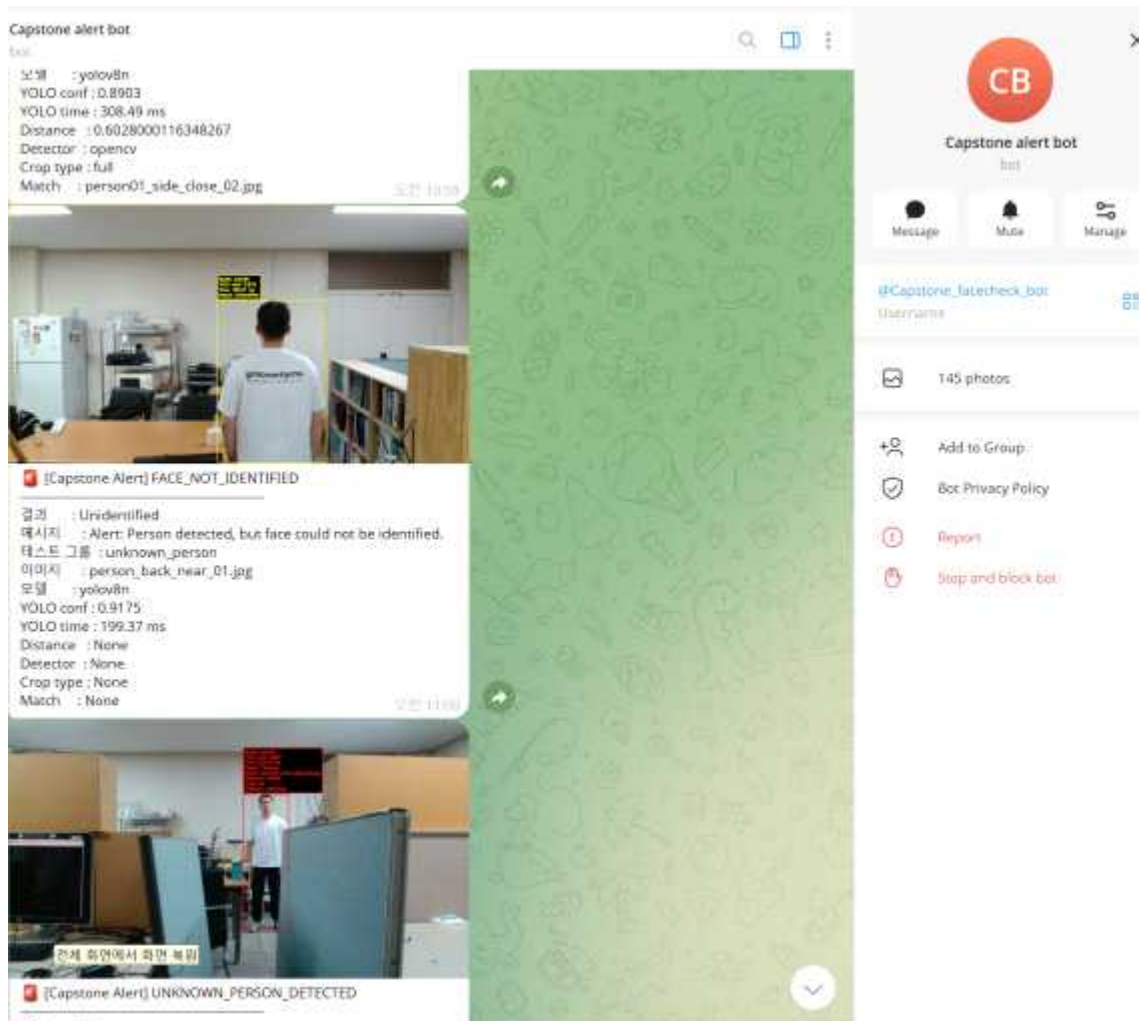
매칭 이미지 : person01_front_close_03.jpg

[결과 파일]

Annotated image : results\annotated\unknown_person\yolov8s\person_side_far_01_yolov8s_annotated.jpg

Crop image : results\crops\unknown_person\yolov8s\person_side_far_01_person1_yolov8s_full.jpg

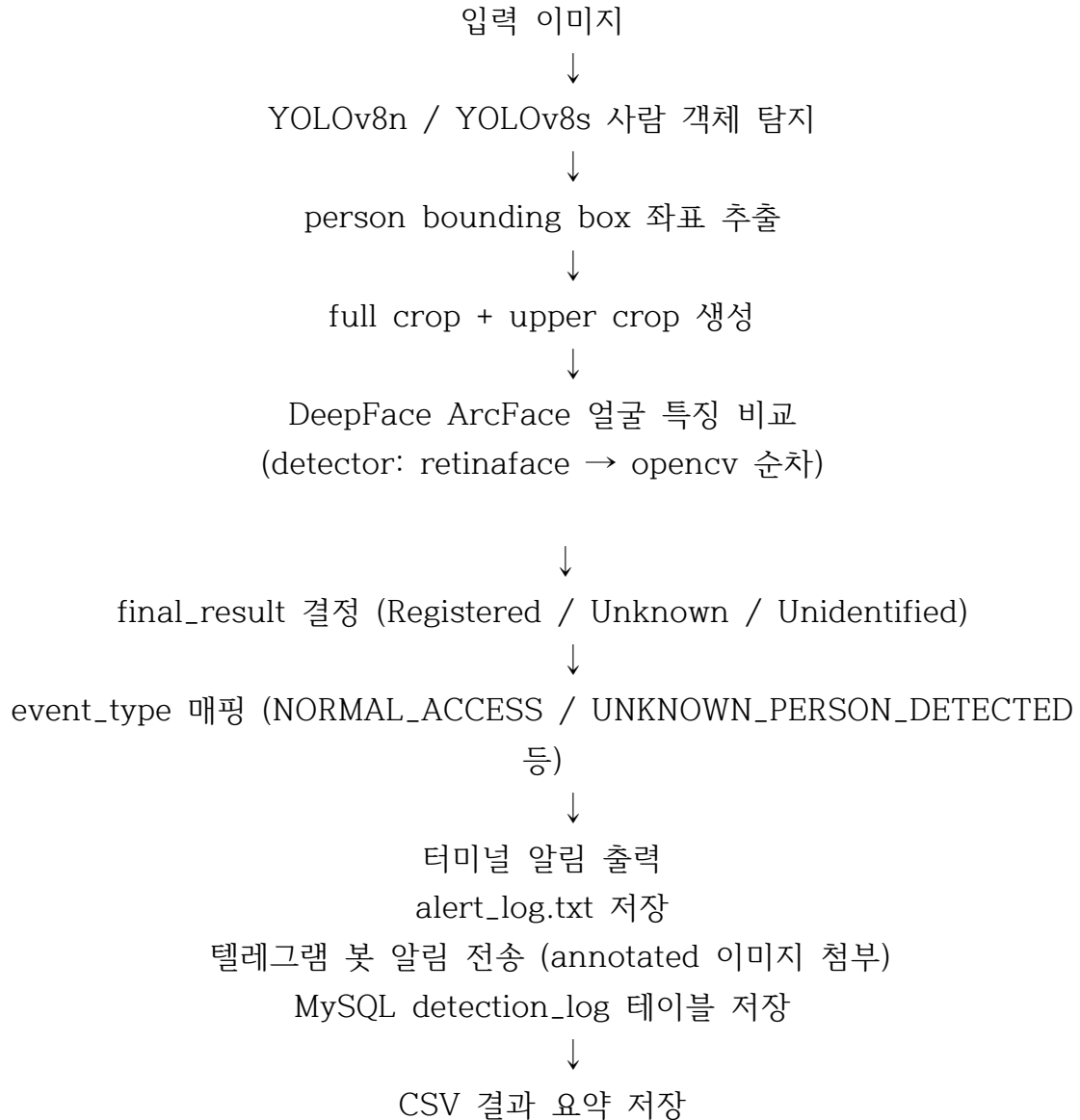
Telegram



또한 알림 발송 내역을 results/alerts/alert_log.txt 파일에 별도로 저장하여, 추후 알림 이력을 텍스트 형태로도 추적할 수 있도록 하였다. 이는 텔레그램 채팅 기록이 손실되는 경우에 대비한 백업 로그 역할을 수행한다.

6. 통합 시스템 동작 흐름

7주차까지 구현된 전체 시스템의 동작 흐름은 다음과 같다.



6주차에서는 흐름이 "final_result 결정 → CSV 저장"으로 끝났던 것에 비해, 7주차에서는 이벤트 매핑 단계와 4가지 후속 처리 단계(터미널 알림, 로그 파일, 텔레그램, DB 저장)가 추가되었다. 이를 통해 단순 판별 시스템에서 이벤트 기반 통합 시스템으로 확장되었다.

7. 실험 결과

7.1 등록 인물 인식 결과

등록 인물 person01에 대해 정면, 반측면, 측면 근거리, 측면 전신 이미지 등 다양한 조건에서 얼굴 인식 결과를 확인하였다. 실험 결과 대부분의 등록 인물 이미지는 DeepFace distance 값이 threshold 0.50보다 낮게 측정되어 Registered로 정상 분류되었다.

측면 얼굴 이미지의 경우 정면 이미지보다 얼굴 특징 추출 조건이 까다로우나, 본 시스템에서는 등록 얼굴 임베딩 캐싱 구조와 얼굴 비교 방식 개선을 통해 측면 얼굴도 정상적으로 비교할 수 있었다. 실험 결과 측면 근거리 이미지는 distance 0.0227 수준으로 낮게 측정되어 등록 인물로 안정적으로 판별되었다.

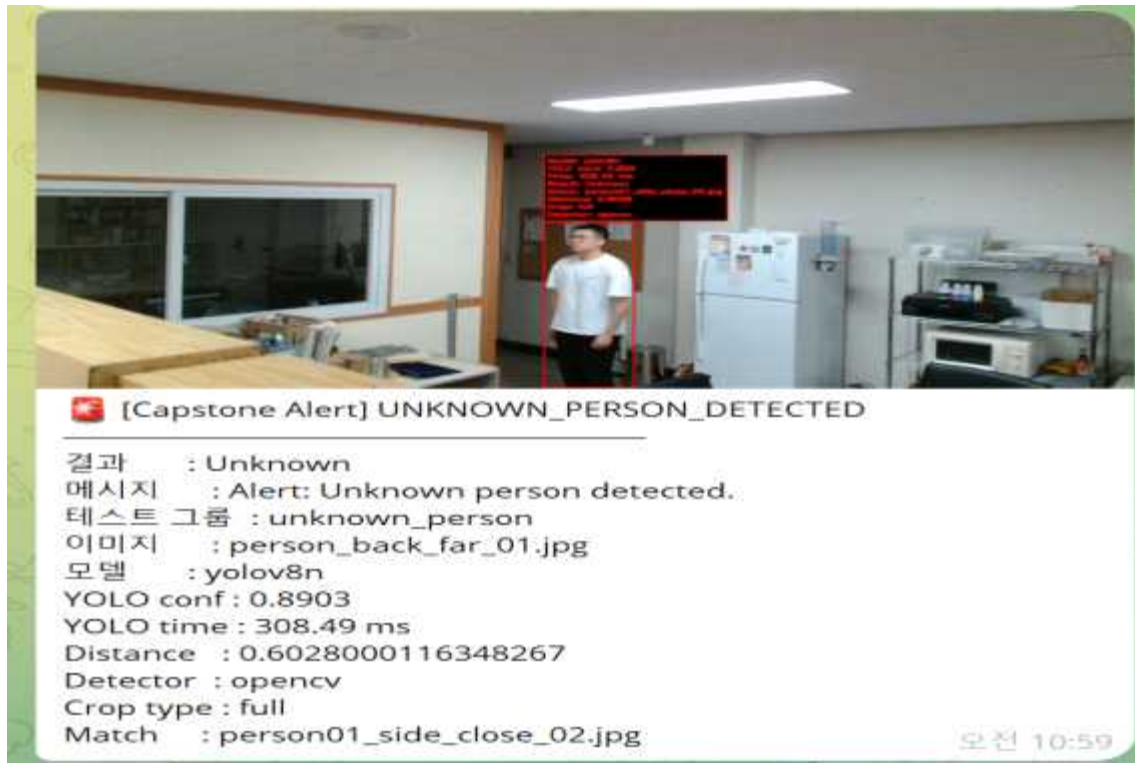
이미지 종류	distance	crop_type	detector	final_result	알림
정면 근거리	약 0.1 ~ 0.3	full	retinaface	Registered	X
반측면 약 45°	0.3708	upper	retinaface	Registered	X
측면 약 90°, 근거리	0.0227	full	retinaface	Registered	X
원거리 전신	0.0117	upper	retinaface	Registered	X

등록 인물로 판별된 경우 final_result는 Registered로 결정되며, 이벤트 유형은 NORMAL_ACCESS로 매핑된다. 이 이벤트는 정상 접근 상황으로 판단되므로 alert_required 값이 False로 설정되어 이메일 및 텔레그램 알림은 발생하지 않고, 결과만 CSV와 MySQL detection_log 테이블에 저장되도록 하였다.

7.2 미등록 인물 탐지 결과

미등록 인물 이미지에 대해서는 DeepFace distance 값이 threshold가 0.50 이상으로 측정되어 Unknown으로 분류되었다. 예를 들어 person_back_far_01.jpg 이미지에서는 YOLOv8n 모델이 사람을 confidence 0.8903으로 탐지하였고, DeepFace 비교 결과 distance가 0.6028로 측정되었다. 이는 설정한 threshold 0.50을 초과하므로 등록 인물이 아닌 미등록 인물로 최종 판별하였다.

Unknown으로 분류된 결과는 get_event_info() 함수를 통해 UNKNOWN_PERSON_DETECTED 이벤트로 매핑되었다. 해당 이벤트는 보안상 확인이 필요한 상황이므로 alert_required=True로 설정되며, 이에 따라 이메일 알림과 텔레그램 알림이 순차적으로 전송되도록 하였다.



7.3 얼굴 식별 실패

사람 객체는 YOLO에 의해 정상적으로 탐지되었지만, DeepFace 단계에서 얼굴 특징을 안정적으로 추출하지 못하는 경우에는 Unidentified로 분류되도록 하였다. 이는 사람이 존재함에도 불구하고 얼굴이 측면을 향하고 있거나, 얼굴 영역이 작거나, 얼굴 검출 조건을 만족하지 못하여 등록 얼굴과의 비교가 불가능한 상황에 해당한다.

예를 들어 person01_standing_side_02.jpg 이미지에서는 YOLOv8s 모델이 사람을 confidence 0.8917로 탐지하였다. 그러나 얼굴 인식 단계에서는 유효한 얼굴 임베딩을 추출하지 못해 distance, detector, crop type 값이 모두 None으로 기록되었다. 이에 따라 최종 결과는 Unidentified로 분류되었고, 이벤트 유형은 FACE_NOT_IDENTIFIED로 매핑되었다.



7.4 실험 결과 종합

7주차 실험을 통해 등록 인물, 미등록 인물, 얼굴 식별 실패 상황에 대한 시스템 동작을 확인하였다. 등록 인물은 threshold 0.50 이하의 distance 값으로 Registered 판별되었고, 미등록 인물은 threshold를 초과하여 Unknown으로 분류되었다. 또한 얼굴 정보가 불명확한 경우에는 Unidentified로 처리하여 보안상 위험한 오판을 줄이도록 하였다.

구분	확인 결과
등록 인물 판별	정면 및 측면 이미지에서 Registered 분류 확인
미등록 인물 판별	threshold 초과 시 Unknown 분류 확인
이벤트 매핑	final_result에 따라 event_type 자동 분류 확인
알림 기능	Unknown 발생 시 이메일·텔레그램 알림 전송 확인
결과 저장	CSV 및 MySQL detection_log 저장 확인
보수적 판별	얼굴 식별 실패 시 Unidentified 처리 가능

결과적으로 7주차 개선을 통해 기존의 단순 얼굴 비교 시스템이 등록 인물 정상 접근, 미등록 인물 감지, 얼굴 식별 실패 상황을 구분하고, 각 상황에 따라 DB 저장과 알림 전송을 수행하는 이벤트 기반 침입 감지 구조로 확장되었음을 확인하였다.

8. 7주차 결과 및 한계

7주차에서는 6주차의 단순 얼굴 비교 구조를 이벤트 기반 침입 감지 시스템으로 확장하여 처리 속도 약 5~10배 단축, 6가지 이벤트 분류 및 MySQL 자동 저장, 텔레그램·이메일 이중 알림 등 세 가지 핵심 기능을 완성하였다.

첫째, 원거리 측면 인식의 한계이다. 5m 이상 떨어진 측면 인물의 경우 얼굴 영역이 작고 픽셀 정보가 부족하여 Unidentified로 분류되었다. 이는 카메라 해상도와 거리의 한계에 따른 것으로, 시스템적 결함이라기보다 입력 데이터의 한계에 해당한다

둘째, 알림 중복 발생 가능성이다. 동일 인물이 연속된 이미지에서 반복 감지될 경우 동일한 이벤트에 대해 이메일과 텔레그램 알림이 중복 발생할 수 있어, 향후 일정 시간 동안 동일 이벤트를 차단하는 쿨다운(cooldown) 로직 적용이 필요하다.

셋째, 이메일 전송 지연 문제이다. Gmail SMTP 기반 이메일 전송은 평균 1~3초가 소요되어, 실시간 영상 처리 환경에서는 병목이 될 가능성이 있다. 향후 threading 기반 비동기 발송 구조로 개선이 필요하다.

넷째, 민감 정보 보안 문제이다. 현재 DB 비밀번호, 텔레그램 봇 토큰, Gmail 앱 비밀번호 등 민감 정보가 코드에 직접 포함되어 있어 보안상 위험이 있다. .env 환경변수 파일로 분리하는 작업이 필요하다.

다섯째, 정적 이미지 기반 처리의 한계이다. 현재 시스템은 정적 이미지 파일을 대상으로 동작하므로, 실제 침입 감지 시스템으로 활용하기 위해서는 라즈베리파이에서 입력되는 영상을 지속적으로 처리할 수 있는 실시간 환경으로 확장해야 한다.

9. 8주차 계획

8주차에서는 7주차에 완성한 이벤트 기반 시스템을 라즈베리파이와 연동하여 실시간 처리 환경으로 확장하는 작업을 진행할 계획이다.

먼저 FastAPI 서버에 /detect POST 엔드포인트를 구현하여 라즈베리파이가 1~2초 주기로 촬영한 이미지를 HTTP로 전송하면 서버가 즉시 처리하여 결과를 JSON으로 반환하는 구조를 구축할 예정이다. 사전 캐싱된 임베딩과 NumPy 기반 거리 계산을 활용하므로 한 프레임당 약 1~2초 내 처리가 가능하여 침입 감지 시나리오에 충분한 실시간성을 확보할 수 있을 것으로 예상된다.

추가로 동일 인물 중복 알림 방지를 위한 30초 쿨다운 로직, 민감 정보 분리를 위한 .env 환경변수 도입, threading 기반 비동기 알림 발송, 라즈베리파이 클라이언트 코드 작성, 그리고 전체 통합 테스트와 시연 영상 제작을 진행하여 1학기 1차 완성본을 도출하는 것을 목표로 한다.